

A Layer of Neurons Is a Matrix

A neuron is a rule that converts a point into a score. Neurons become the rows of a matrix, and applying them to the data becomes one product.

This meeting, in one sentence

A layer of neurons is a matrix, and applying it to data is one product: written in NumPy, written again in PyTorch, run on a GPU, and counted.

- Meeting 10 stored data as a matrix X with one point in each row. Today a **neuron**, a rule that converts a point into a score, is stored as a row of a matrix W .
- Three acts: the layer in NumPy, on the pixels of your group's image; the same layer in PyTorch and on a GPU; what the product costs.
- For each act: slides, then a Colab block in the class notebook, then a post. **Ask Colab for a GPU before you run anything:** the class notebook says how.

One neuron, one point

A **neuron** has one **weight** for each coordinate of a point, and one more number, its **bias**. On a point x with m coordinates it computes the **score**

$$s = w_1x_1 + w_2x_2 + \dots + w_mx_m + b$$

- The neuron **fires** when $s \geq 0$. Every coordinate is multiplied by its weight; the products and the bias are added.
- Today a point is one **pixel** of the group's image, with $m = 3$ coordinates: red, green, blue. The grey rule of meeting 10 is the neuron with the weights 0.299, 0.587, 0.114 and the bias 0.

```
the pixel    x = [0.56 0.39 0.1 ] (red, green, blue)
the weights w = [0.299 0.587 0.114] and the bias b = 0.0
```

The same score, with matrices

$$s = w^T x + b$$

- In linear algebra x and w are **columns**, $m \times 1$. The **transpose** w^T holds the same numbers as a row, $1 \times m$. A row times a column is one number: the sum of the products of matching entries.
- The shapes say why the transpose is there: $(1 \times m)(m \times 1)$ is 1×1 . Without it, $(m \times 1)(m \times 1)$ is not defined.
- In NumPy, `w_grey` and `x` have **one** dimension, shape `(3,)`: neither a row nor a column. `@` between them is their dot product, and such an array is its own transpose.

```
s_one = w_grey @ x + b_grey          # the dot product of the weights and the point, plus the bias
print(f"s = {s_one:.3f}")
print("shape of w_grey:", w_grey.shape, "  shape of x:", x.shape)
```

```
s = 0.411
shape of w_grey: (3,)  shape of x: (3,)
```

One neuron, many points

```
X = image.reshape(-1, 3)           # one row for each pixel
n, m = X.shape
print(f"shape of X: {X.shape}, which is n = {n:,} points with m = {m} coordinates")

s_grey = X @ w_grey + b_grey       # the scores of the grey neuron on all the points
print("shape of X @ w_grey + b_grey:", s_grey.shape)
```

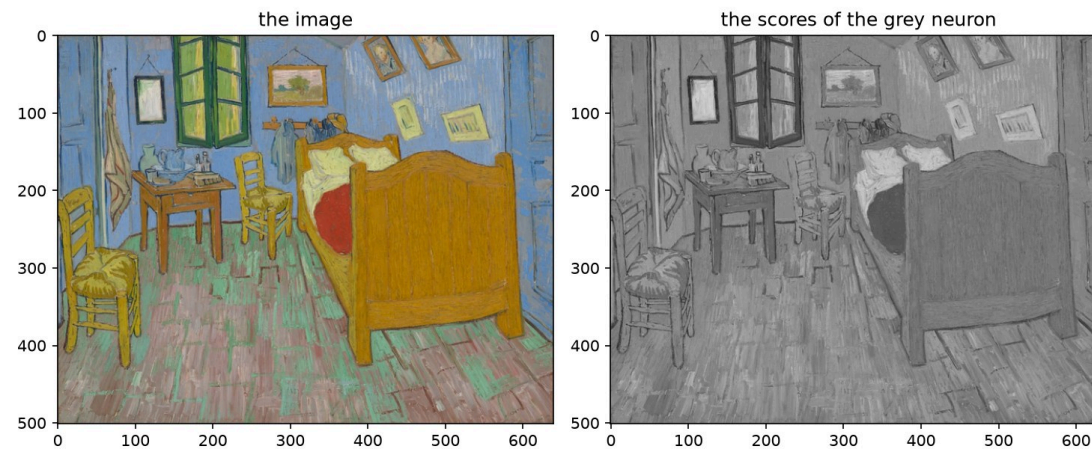
```
shape of X: (320640, 3), which is n = 320,640 points with m = 3 coordinates
shape of X @ w_grey + b_grey: (320640,)
the scores of the first five points: [0.482 0.532 0.5    0.482 0.477]
```

- `image.reshape(-1, 3)` writes the pixels as the rows of X: n points with m = 3 coordinates, n = height × width.
- `X @ w_grey` is one score for each point, n numbers. `+ b_grey` adds one number to all of them: **broadcasting**, as `v + 10` in meeting 10.

The same scores, with matrices

$$s = Xw + b \mathbf{1}$$

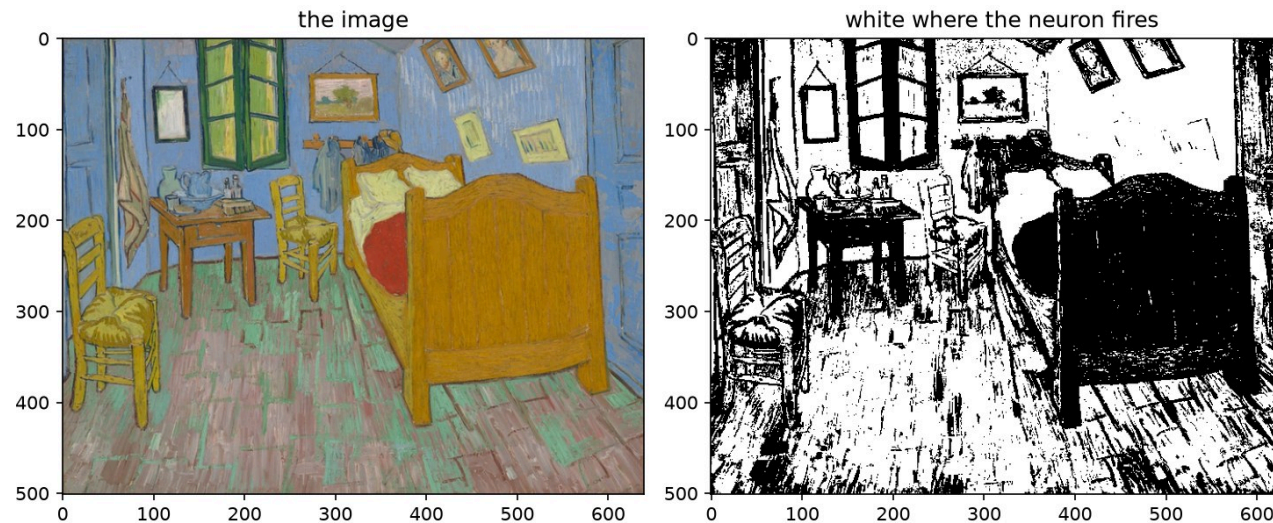
- X is $n \times m$ and w is $m \times 1$, so Xw is $n \times 1$. Entry i is row i of X times w . Row i of X is the point x_i written as a row, x_i^T , and $x_i^T w$ is the same number as $w^T x_i$. **The transpose has moved from the weights to the point, because the data stores each point as a row.**
- $\mathbf{1}$ is a column of n ones: the mathematics needs it in order to add the single number b to each of the n scores. NumPy's broadcasting does the same.



A neuron that fires

```
b_bright = -s_grey.mean() # minus the mean brightness of the image
fires = X @ w_grey + b_bright >= 0 # True for the points on which the neuron fires
print(f"bias {b_bright:.3f}: the neuron fires on {fires.mean():.1%} of the {n:,} pixels")
```

bias -0.518: the neuron fires on 53.1% of the 320,640 pixels



A brightness is never negative, so with the bias 0 the grey neuron fires on every pixel. With the bias $-(\text{mean brightness})$ it fires on the pixels brighter than the average. $\theta = -b$ is the **threshold** of the neuron.

Two ways to store the points

```
by_rows = X @ w_grey + b_bright      # points as rows: X has shape (n, m)
by_columns = w_grey @ X.T + b_bright  # points as columns: X.T has shape (m, n)
```

shape of X: (320640, 3) shape of X.T: (3, 320640)

The two results are equal: True

`w_grey @ X` fails: `w_grey` has 3 entries and X has 320,640 rows, so the shapes do not fit

- In this course, as in pandas, NumPy and PyTorch, a point is a **row** of X, and the scores are $Xw + b$.
- Many books store a point as a **column**. Their data matrix is X^T , $m \times n$, and the same scores are written $w^T X + b$.
- `w_grey @ X` fails: 3 entries against n rows. The shapes do not fit.

Many neurons, one point

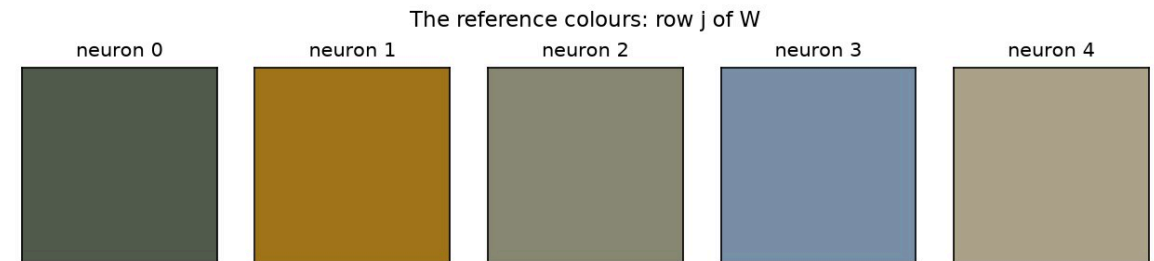
A **layer** is a set of K neurons that read the same point. **Row j of W holds the weights of neuron j** , as a row of X holds a point, and **b** holds the K biases. The K scores of a point are **$s = Wx + b$** : W is $K \times m$ and x is $m \times 1$, so Wx is $K \times 1$, and so is b . Nothing is repeated.

```
K = 5
C = reference_colours(X, K)
W = C.copy()
b = -0.5 * (C ** 2).sum(axis=1)
```

```
# K rows and 3 columns: the reference colours, darkest first
# the weights of the layer: row j is reference colour j
# the biases of the layer: one number for each neuron
```

shape of W : (5, 3) shape of b : (5,)

```
W =
[[0.32 0.36 0.31]
 [0.63 0.46 0.1 ]
 [0.53 0.53 0.45]
 [0.48 0.57 0.66]
 [0.67 0.65 0.55]]
b = [-0.16 -0.31 -0.38 -0.49 -0.59]
```



Five **reference colours** of the image, found by a method called k-means. Neuron j has the weights c_j and the bias $-\frac{1}{2}|c_j|^2$: **the largest score belongs to the neuron whose colour is nearest to the pixel.**

The scores of one pixel

$$s_x = W @ x + b$$

the K scores of the pixel x

the pixel $x = [0.56 \ 0.39 \ 0.1 \]$

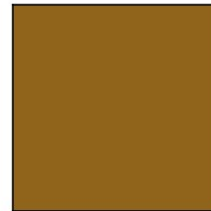
shape of $W @ x + b$: (5,)

the scores of the 5 neurons: $[0.19 \ 0.237 \ 0.17 \ 0.065 \ 0.104]$

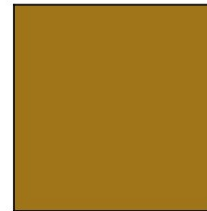
The largest score is that of neuron 1, whose colour is $[0.63 \ 0.46 \ 0.1 \]$.

The pixel, and the colour with the largest score

the pixel x



colour of neuron 1



Positions are counted from 0, so the neurons are numbered 0 to 4. The neuron with the largest score is the nearest colour.

Many neurons, many points

```
def layer_loops(X, W, b):  
    """The scores of the K neurons of a layer on the n points of X, one multiplication at a time."""  
    n, m = X.shape  
    K = W.shape[0]  
    S = np.empty((n, K))  
    for i in range(n):           # every point  
        for j in range(K):       # every neuron  
            total = b[j]  
            for c in range(m):    # every coordinate  
                total += W[j, c] * X[i, c]  
            S[i, j] = total  
    return S
```

$$S = XW^T + 1b^T$$

- S has n rows and K columns: **row i belongs to point i, column j to neuron j**. The loops compute it from the definition.
- X is $n \times m$ and W^T is $m \times K$, so XW^T is $n \times K$. **W is transposed because a neuron is a row of W, and the product needs each neuron's weights as a column.** $1b^T$ is the biases written again on every row: broadcasting. **In Colab you write the three loops as one line.**

Loops against the product

`n = 320,640 points, m = 3 coordinates, K = 5 neurons`

`The innermost line of the loops runs  $n \times K \times m = 4,809,600$  times.`

version	seconds
---------	---------

three loops	1.0256
-------------	--------

one product	0.0058
-------------	--------

`The loops take 176 times as long as the product.`

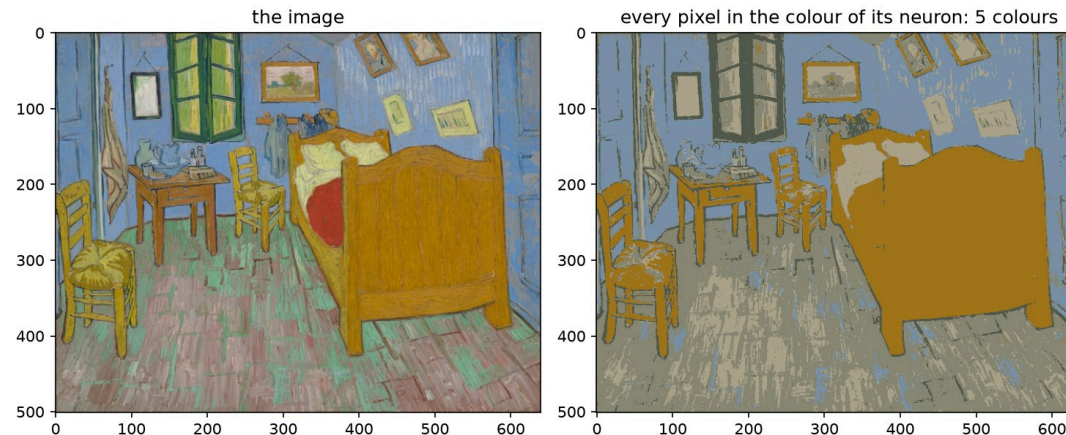
- The innermost line of the loops runs $n \times K \times m$ times. Act 3 comes back to that number.
- The product carries out the same multiplications and additions, inside NumPy.

The neuron with the largest score

```
winner = S.argmax(axis=1)
```

for each point, the neuron with the largest score

```
shape of winner: (320640,)    its first ten entries: [2 2 2 2 2 2 2 2 2 2]
neuron 0, colour [0.32 0.36 0.31]: the largest score on 7.8% of the pixels
neuron 1, colour [0.63 0.46 0.1 ]: the largest score on 23.0% of the pixels
neuron 2, colour [0.53 0.53 0.45]: the largest score on 33.1% of the pixels
neuron 3, colour [0.48 0.57 0.66]: the largest score on 24.4% of the pixels
neuron 4, colour [0.67 0.65 0.55]: the largest score on 11.7% of the pixels
```

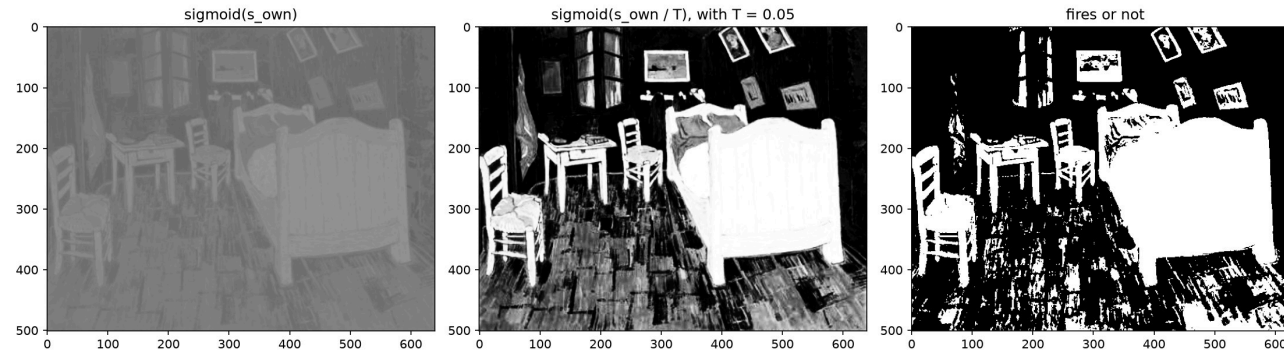


When each neuron stands for one possible answer, the answer for a point is the neuron with the largest score: the **argmax** of its row. `C[winner]` takes, for every point, the colour of that neuron.

The sigmoid, a function of every entry

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

```
def sigmoid(s):  
    """The sigmoid of every entry of the array s."""  
    return 1 / (1 + np.exp(-s))
```



- $\sigma(0) = 0.5$; large positive scores give nearly 1, large negative scores nearly 0.
`1 / (1 + np.exp(-s))` acts on **every entry**: pointwise, and the result keeps the shape of its input.
- Divided by a **temperature** T first, the scores spread out: a small T brings the sigmoid close to fires-or-not.

The softmax, a function of every row

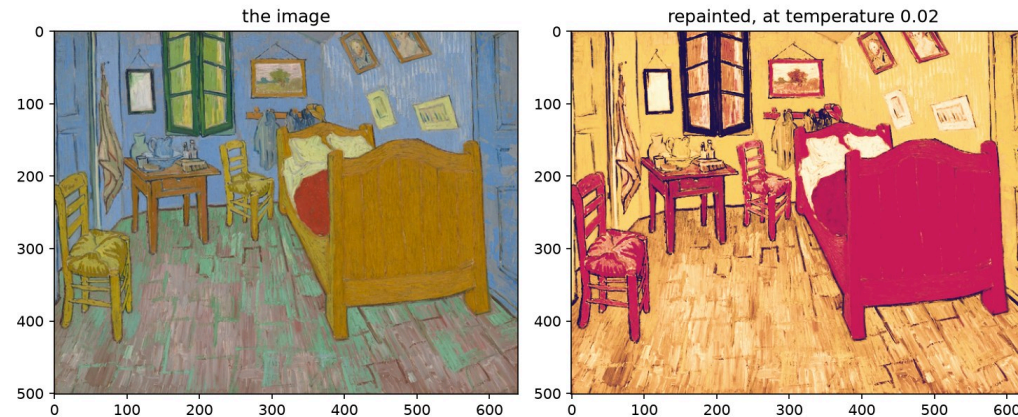
$$p_k = \frac{e^{s_k}}{e^{s_1} + e^{s_2} + \dots + e^{s_K}}$$

```
def softmax_rows(S):  
    shifted = S - S.max(axis=1, keepdims=True)    # the largest entry of every row becomes 0  
    E = np.exp(shifted)  
    return E / E.sum(axis=1, keepdims=True)
```

- The K scores of a point become K positive numbers that add to 1, read as probabilities. Every entry of the result depends on **all the entries of its row**: the softmax is a function of a row, the sigmoid of an entry.
- Written by an AI assistant to stated requirements: no loop, no library's softmax, and the largest entry of each row subtracted first. Adding one number to all the scores of a point leaves its probabilities as they were; after the subtraction no exponential exceeds 1.
- `axis=1`: along each row. `keepdims=True`: the largest entry and the sum stay a column, shape (n, 1), and broadcast against the n rows.

Repaint the image: a small network

```
Prob = softmax_rows(layer(X, W, b) / T)    # n rows and K columns  
new = Prob @ PAINT                        # n rows and 3 columns
```



- Each neuron gets a **paint**, a colour of the group's choice: the rows of `PAINT`, $K \times 3$.
`Prob @ PAINT` is $n \times 3$: the new colour of a pixel is a mixture of the K paints, each counted with the probability of its neuron.
- The second product is of the same kind as the first: n points, now with K coordinates, and three neurons. **Two layers with a function between them are a small neural network**, applied to all the pixels at once.

The same line on other data

```
W_seasons = np.array([[ -1.35750, -1.96665],      # the neuron of winter
                      [  0.54402, -1.21157],      # the neuron of spring
                      [  0.92758,  3.07303],      # the neuron of summer
                      [-0.11410,  0.10518]])      # the neuron of fall
b_seasons = np.array([-0.64726, 1.19295, -1.73890, 1.19321])

S_days = Z_days @ W_seasons.T + b_seasons        # the line of Step 4
```

shape of Z_days: (1457, 2) shape of W_seasons: (4, 2)

shape of S_days: (1457, 4)

The neuron with the largest score names the right season on 59.3% of the 1,457 days.

- A layer of four neurons, one for each season, reads the $m = 2$ coordinates of a day at Newark airport: its high and its low temperature. Nobody chose these weights by hand: a program adjusted them in many small steps until the layer named the right season for as many days of 2006 to 2017 as it could. That is **training**.
- In meeting 10, `u @ v` was the same line with one neuron: the vector of a word as the weights, applied to 50,000 points.

Now in Colab: Act 1

- The class notebook, with the GPU asked for, the setup run and your group's number typed, as at the start.
- **Act 1:** run the cell that builds the layer of five neurons on your image. Then **write the one line** of the function `layer`: the three loops as one product. The check tells you which array to transpose.
- Repaint your image: choose five paints, and try the temperatures 1, 0.1 and 0.002.
- **Post** on the discussion board: the printed line, the picture of your image beside the repainted one with one sentence on what it shows, and one sentence on what the temperature changed. One member posts; begin with your group's number and the names of all its members.

The four cases

	one point x , shape $(m,)$	n points X , shape (n, m)
one neuron: w of shape $(m,)$, b a number	$w @ x + b$ is a number	$X @ w + b$ has shape $(n,)$; b is broadcast
K neurons: W of shape (K, m) , b of shape $(K,)$	$W @ x + b$ has shape $(K,)$	$X @ W.T + b$ has shape (n, K) ; b is broadcast

- The bias needs help exactly when there are many points: the mathematics writes a column of ones, and NumPy broadcasts.
- The shapes do not always warn of a missing transpose: when $K = m$, $X @ W + b$ runs, and its numbers are wrong.
- Act 2 writes the same line in PyTorch.